Reverse Engineering: Encoding

2026-10-13

Table of contents

1 Introduction **1**

2 Download the File **2**

3 Open in Cutter **2**
3.1 Search the Strings Panel 2
3.2 Locate Main 2

4 Examine the Decode Function **3**
4.1 Understand the Loop 3

5 Find the Encoded Data **4**

6 Recover the Message **4**

7 Verify Your Finding **5**

1 Introduction

Malware authors frequently hide strings — command-and-control addresses, passwords, error messages — by encoding them so they do not appear in the binary as readable text. A basic search for strings will come up empty, which is exactly the intent. Analysts must instead understand the encoding routine from assembly and reverse it manually to recover the hidden content.

In this lab you will analyze a binary that contains a message that is never stored as plaintext. You will use Cutter to locate the encoding scheme, identify the key, find the encoded data, and recover the original message — all without running the program.

The file needed for this lab is located on Canvas as `encoding-executables.zip`. It is recommended that you complete this lab on your virtual machine.

> **Do Not Run the Binary First**
>
> Treat this binary as you would suspected malware — analyze it statically before executing it. Running it first defeats the purpose of this lab. The verification step at the end is where you confirm your findings by running it.

2 Download the File

Download `encoding-executables.zip` from Canvas onto your VM and unzip it. Inside you will find folders for each architecture (`x86-64` and `arm64`). Use the folder that matches your VM. Make the file executable:

```
chmod +x encode
```

3 Open in Cutter

Open the binary in Cutter. At the `Load Options` dialog, leave the defaults and click `Ok`.

3.1 Search the Strings Panel

Navigate to the `Strings` tab. This panel lists every sequence of printable characters in the binary that is long enough to likely be a meaningful string.

Look through the results. You will find standard library strings and short fragments, but **you will not find the hidden message**. This is the whole point — the message has been transformed so that it does not appear as readable text in the binary.

Make note of what you do and do not find here. When analysts claim a binary "has no interesting strings," this is what they mean.

3.2 Locate Main

Navigate to the `Graph` tab and select `main` from the left panel or the `Functions` panel.

`main` is short. It calls two functions and then exits. One call is to a standard library function (`printf` or similar). The other is to a function defined within the binary itself — this is where the interesting work happens.

Follow the call to the non-library function.

4 Examine the Decode Function

You are now looking at the function that processes the hidden data. Analyze it in both the **Graph/Disassembly** view and the **Decompiler** view. Switching between them will help — the assembly shows you exactly what instructions execute, while the decompiler gives you a higher-level view of the logic.

> Opening the Decompiler
>
> With the function visible in the Graph or Disassembly view, select the `Decompiler` tab at the bottom of the screen. If the tab is not visible, enable it from the `Windows` menu. The decompiler panel shows a C-like approximation of the function's logic. It is not always perfectly accurate, but it is a useful starting point.

4.1 Understand the Loop

The function contains a loop that iterates over every byte in a buffer. Inside the loop, each byte is modified by a single operation using a constant value.

Identify:

1. **What operation is applied to each byte?** (Look for an instruction that combines a byte from the buffer with an immediate constant. In x86-64, this will appear as an instruction operating on a register like `al`. On arm64, it will operate on a register like `w8` or `w9`.)
2. **What is the constant value used?** (It will appear as a hexadecimal immediate in the instruction — something like `0x??`.)
3. **What is the length of the buffer?** (The length is passed as an argument to this function. In the decompiler view, it will appear as a parameter or a local variable used as the loop bound. In x86-64 assembly, the second argument is in `esi` at the call site; on arm64 it is in `w1`.)

> **XOR and Its Inverse**
>
> XOR (`^`) is its own inverse: if `A ^ K = B`, then `B ^ K = A`. This means the same operation that encodes data also decodes it. If you know the key and the encoded bytes, you can recover the original by applying XOR again.

5 Find the Encoded Data

Now you need to find the actual bytes that the function operates on. These are the encoded contents of the hidden message.

Go back to the call site in `main` where this decode function is called. Look at the instruction that loads the first argument (the buffer address). In x86-64, this will be a `lea rdi, [...]` or `mov rdi, ...` instruction loading an address. On arm64, it will load into `x0`.

Double-click the data address referenced in that instruction to navigate to that address, then switch to the `Hexdump` tab (as you did in the Cutter lab, you can also paste the address into the search bar at the top). You should see a row of bytes beginning at that address. Read off the bytes for the length you identified earlier.

Write them down as a list. They will look like a series of hex values with no obvious pattern.

> **Reading the Hexdump**
>
> In Cutter's Hexdump tab, bytes are displayed in rows of 16. The address on the left tells you where each row starts. Identify the starting address of the encoded array and read off exactly as many bytes as the length you found. Ignore any bytes after that count.

6 Recover the Message

You now have everything you need:

- The operation applied to each byte (likely XOR)
- The key constant
- The list of encoded bytes

Apply the inverse operation to each byte to recover the original message. The easiest way to do this is with a short Python one-liner in your terminal. Replace the byte list and key with what you found:

```
python3 -c "
data = [0x??, 0x??, ...]
print(''.join(chr(b ^ 0x??) for b in data))
"
```

The output should be a readable English string.

7 Verify Your Finding

Now run the binary to confirm your result:

```
./encode
```

The program decodes the message at runtime and prints it. Compare the output to what you recovered through static analysis — they should match exactly.

If they do not match, revisit the key constant and the byte list. A single mistyped or transposed digit in one of the hex values is the most common mistake.